Programming Fundamentals

Week 6A – Working with Data

Programming Fundamentals Team

SETU

2026-10-08

Where are we now?

We already know **lists**, **dictionaries**, **functions**, **selection** and **loops**.

Today we combine them to solve a bigger problem.

> Our focus is not new syntax. It's deciding how to organise and work with data.

A familiar collection

```
students = [
    {"id": 1, "name": "Aoife", "mark": 72},
    {"id": 2, "name": "Ben", "mark": 48},
    {"id": 3, "name": "Cara", "mark": 65}
]
```

Question: What is the list? What is each dictionary?

Reading the records

```
for student in students:
    print(student["name"], student["mark"])
```

Predict: What appears on the screen?

Discuss: Why use a loop instead of three print statements?

Selection while processing

```
for student in students:
    if student["mark"] >= 40:
        print(student["name"], "Pass")
    else:
        print(student["name"], "Fail")
```

The loop visits each record; selection makes a decision about it.

Counting and totals

```
passes = 0
total = 0
for student in students:
    total += student["mark"]
    if student["mark"] >= 40:
        passes += 1
print("Passes:", passes)
print("Average:", total / len(students))
```

Question: What if the list is empty?

Finding one record

```
def find_student(students, student_id):
    for student in students:
        if student["id"] == student_id:
            return student
    return None
```

Why is `return None` after the loop rather than inside it?

What is CRUD?

Create — add a record.

Read — display or find records.

Update — change a record.

Delete — remove a record.

CRUD describes what our application needs to do, not a particular Python command.

Design before code

Imagine a small Dog Day Care Manager.

Each dog has an ID, name, breed and age.

For each CRUD operation, discuss:

- What information do we need?
- What do we find or change?
- What could go wrong?

A useful pattern

```
def find_dog(dogs, dog_id):
    for dog in dogs:
        if dog["id"] == dog_id:
            return dog
    return None
```

Think: Which CRUD operations could reuse `find_dog()`?

A bigger question

We've used lists and dictionaries to manage **dogs** and **students**.

Could we use the same techniques to manage **knowledge**?

This is the connection to our next project.

Our ongoing project

PC Troubleshooting Assistant

The program asks questions about a computer problem, then recommends something to try.

Example:

- Does the computer turn on? **No**
- Is the power cable connected? **Yes**
- Is the power light on? **No**

Recommendation: **Check the power supply or socket.**

Where does its knowledge live?

```
questions = [
    {"id": 1, "text": "Does the PC turn on?"},
    {"id": 2, "text": "Is the cable connected?"},
    {"id": 3, "text": "Is the power light on?"}
]
```

We can **store questions as data** rather than scattering them throughout our code.

Even rules can be data

Even rules can be data

```
rules = [
    {
        "id": 1,
        "question_id": 1,
        "expected_answer": "no",
        "recommendation": "Check the power supply"
    }
]
```

Today: understand the structure. **Later:** write code to evaluate rules.

CRUD for an expert system

Create — add a troubleshooting question.

Read — list all questions.

Update — correct the wording of a question.

Delete — remove an outdated question.

The same CRUD ideas work in very different applications.

Try it together

```
def display_questions(questions):
    for question in questions:
        print(question["id"], question["text"])
```

Challenge: Design `find_question(questions, question_id)`.

What should it return if the ID does not exist?

Looking ahead

Week 6: Questions and rules as **lists and dictionaries**.

Week 7: Save the knowledge base using **JSON**.

Later: Add **exceptions**, **classes**, **inheritance** and a general rule engine.

> We are not building the whole expert system today. Each week adds one piece.